

React Router - Grund

Steg-för-steg för en grundläggande routing-applikation

Hur använder **React Router** för att växla mellan en startsida och en infosida. Koden är helt avskalad från styling för att vi ska kunna fokusera till hundra procent på hur själva mekaniken fungerar.

1. Nödvändiga Importer

Först och främst måste vi hämta de verktyg vi behöver för att applikationen ska fungera. React laddas in som vanligt, och sedan hämtar vi fyra specifika verktyg från det externa biblioteket `react-router-dom`.

```
import React from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
```

- **BrowserRouter:** Huvudmotorn som håller koll på webbläsarens adressfält och historik.
- **Routes:** En container som ser till att *endast en* rutt (sida) visas åt gången.
- **Route:** Själva regeln som kopplar ihop en specifik URL-adress med en React-komponent.
- **Link:** Reacts smarta version av en vanlig HTML-länk (`<a>`) som förhindrar att sidan laddas om.

2. Våra Sidkomponenter (Vyer)

I React finns det ingen skillnad rent kodmässigt på en "Knapp-komponent" och en "Sid-komponent". Våra två sidor, **Hem** och **Om oss**, är helt vanliga JavaScript-funktioner som returnerar lite HTML (JSX). Det är först senare, när vi kopplar dem till routern, som de börjar agera som faktiska sidor.

```
function Home() {
  return (
    <div>
      <h2>Hem-sidan</h2>
      <p>Välkommen till startsidan för vårt enkla router-exempel!</p>
    </div>
  );
}

function About() {
  return (
    <div>
      <h2>Om oss</h2>
      <p>Detta är en enkel sida som visar hur React Router fungerar.</p>
    </div>
  );
}
```

3. App-komponenten (Själva stommen)

Nu sätter vi ihop vår huvudkomponent App. Den absolut viktigaste regeln här är att all kod som har med routing, länkar eller sidbyten att göra **måste vara omsluten av** `<BrowserRouter>`. Utan denna motor kraschar programmet om den stöter på en länk.

```
export default function App() {
  return (
    {/* Startar routern och övervakar adressfältet */}
    <BrowserRouter>
      <div>
        <h1>React Router Demo</h1>

        {/* -- NAVIGERING OCH RUTTER LÄGGS HÄR (se nedan) -- */}

      </div>
    </BrowserRouter>
  );
}
```

4. Navigationsmenyn (Länkar)

Inuti vår App bygger vi en enkel meny. Om vi hade använt klassisk HTML () hade webbläsaren laddat om hela fönstret vid ett klick. Genom att använda <Link to="..."> byter React bara mjukt ut innehållet på skärmen utan att vi märker något "blink".

```
<nav>
  <Link to="/">Hem</Link> | <Link to="/about">Om oss</Link>
</nav>

<hr /> {/* En enkel linje för att separera menyn från innehållet */}
```

5. Regelverket (Routes)

Under menyn och linjen placerar vi våra `<Routes>`. Det är denna del av koden som fungerar som en växel. När adressfältet i webbläsaren ändras tittar Routes igenom sina Route-regler från topp till botten. När den hittar en sökväg (path) som stämmer överens, ritar den ut den motsvarande komponenten i egenskapen `element`.

```
{/* Vaktar rutterna och visar rätt innehåll */}
<Routes>
  {/* Om URL slutar på "/" -> Visa <Home /> */}
  <Route path="/" element={<Home />} />

  {/* Om URL slutar på "/about" -> Visa <About /> */}
  <Route path="/about" element={<About />} />
</Routes>
```

Sammanfattningsvis: När användaren klickar på länken "Om oss", uppdateras adressfältet till "/about". BrowserRouter upptäcker detta, signalerar till Routes som hittar matchningen, och komponenten <About /> trycks in på skärmen!